

University OF Engineering and Technology, Lahore
Computer Engineering Department

Course Name: PF	Course Code: CMPE-112L
Assignment Type: lab	Dated: 27 APRIL 2026
Semester: 1st	Session: Spring 2026
Lab/Project/Assignment #: 7	CLOs to be covered: CLO3
Lab Title: List, Tuple	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 12

Lab Goals/Objectives:

1. Objective of Lists

- **Dynamic Data Management:** The primary objective is to hold a collection of data that can dynamically grow, shrink, or change during program execution.
- **Mutability (In-Place Modifications):** Lists are designed to let you add elements (`append()`), remove specific elements (`remove()`), or swap out items at specific indices without needing to recreate the entire data structure in memory.
- **Handling Heterogeneous/Mixed Data:** Lists allow you to group completely different data types (like mixing strings and integers together) into a single, cohesive sequential unit.

2. Objective of Tuples

- **Data Integrity & Write-Protection:** The main objective of a tuple is **immutability**. It ensures that once a collection of data is defined, it cannot be modified, overwritten, appended to, or tampered with accidentally by other parts of the script.
- **Representing Fixed Structures:** Tuples are meant for grouping data sets that belong together as a permanent record (e.g., coordinates, database records, or constant settings).
- **Performance Optimization:** Because they cannot change, Python handles tuples much faster and with less memory overhead than lists, making them highly efficient for read-only tracking.

Equipment Required:

- Computer/Laptop
- Python 3.x
- VS Code / PyCharm / IDLE

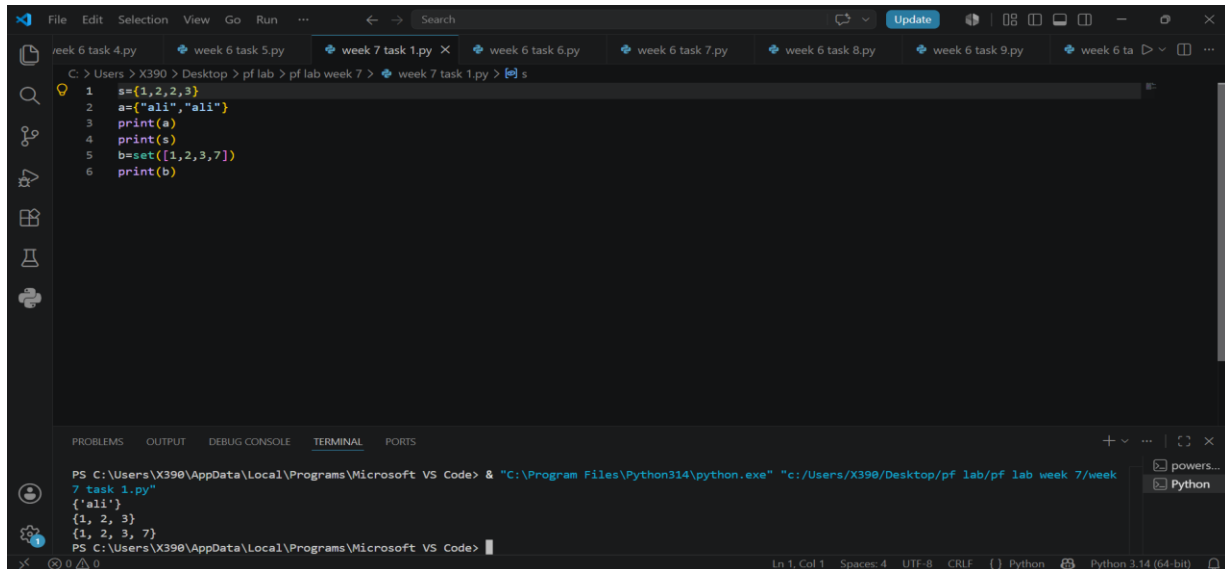
THEORY:

In Python, a **list** is a mutable, ordered collection of items enclosed in square brackets `[]` that allows you to add, remove, or modify elements dynamically after creation. Conversely, a **tuple** is an immutable, ordered collection defined by parentheses `()` whose elements cannot be altered, appended, or deleted once initialized. Both data structures support indexing, slicing, and can store heterogeneous data types, including duplicate values. The critical difference lies in memory efficiency and data safety: tuples are faster and consume less memory because of their fixed structure, making them ideal for read-only data. Lists are preferred when the dataset requires frequent modifications, acting as dynamic arrays, whereas tuples are often used to protect data integrity from accidental changes.

TASK NO 1:

Write a program that demonstrates how to initialize sets in Python and highlights their ability to automatically eliminate duplicate elements

RESULT:



The screenshot shows a VS Code editor with a file named 'week 7 task 1.py'. The code in the editor is as follows:

```
1 s={1,2,2,3}
2 a={"ali","ali"}
3 print(a)
4 print(s)
5 b=set([1,2,3,7])
6 print(b)
```

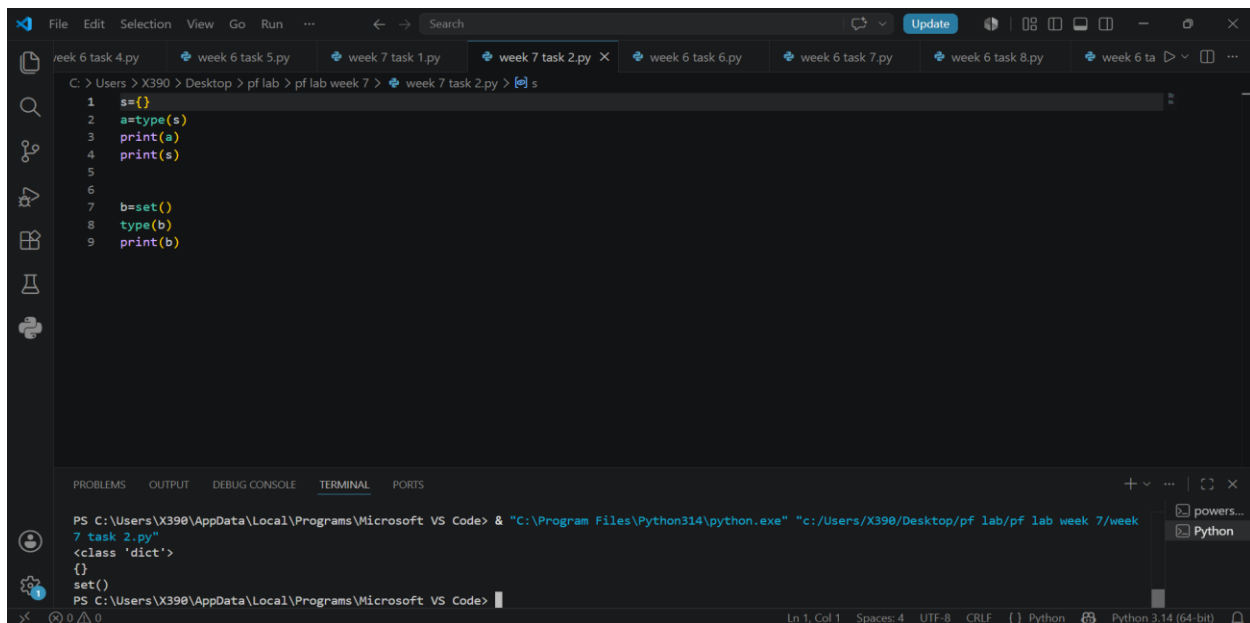
The terminal output at the bottom shows the execution of the script:

```
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf lab/pf lab week 7/week 7 task 1.py"
{'ali'}
{1, 2, 3}
{1, 2, 3, 7}
```

TASK NO 2:

Write a program to demonstrate the syntactical distinction between initializing an empty dictionary and an empty set in Python using built-in methods

RESULT:



The screenshot shows a VS Code editor with a file named 'week 7 task 2.py'. The code in the editor is as follows:

```
1 s={}
2 a=type(s)
3 print(a)
4 print(s)
5
6
7 b=set()
8 type(b)
9 print(b)
```

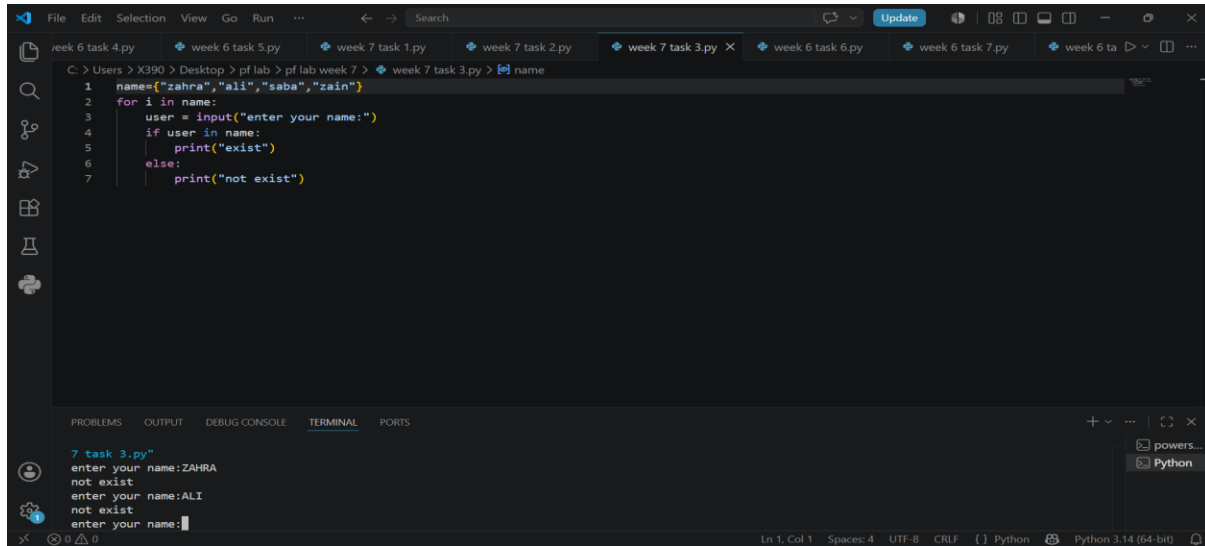
The terminal output at the bottom shows the execution of the script:

```
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf lab/pf lab week 7/week 7 task 2.py"
<class 'dict'>
{}
set()
```

TASK NO 3:

Write a program that uses a loop to repeatedly prompt for user input and utilizes the `in` operator to check if the entered string values exist inside a predefined set.

RESULT:



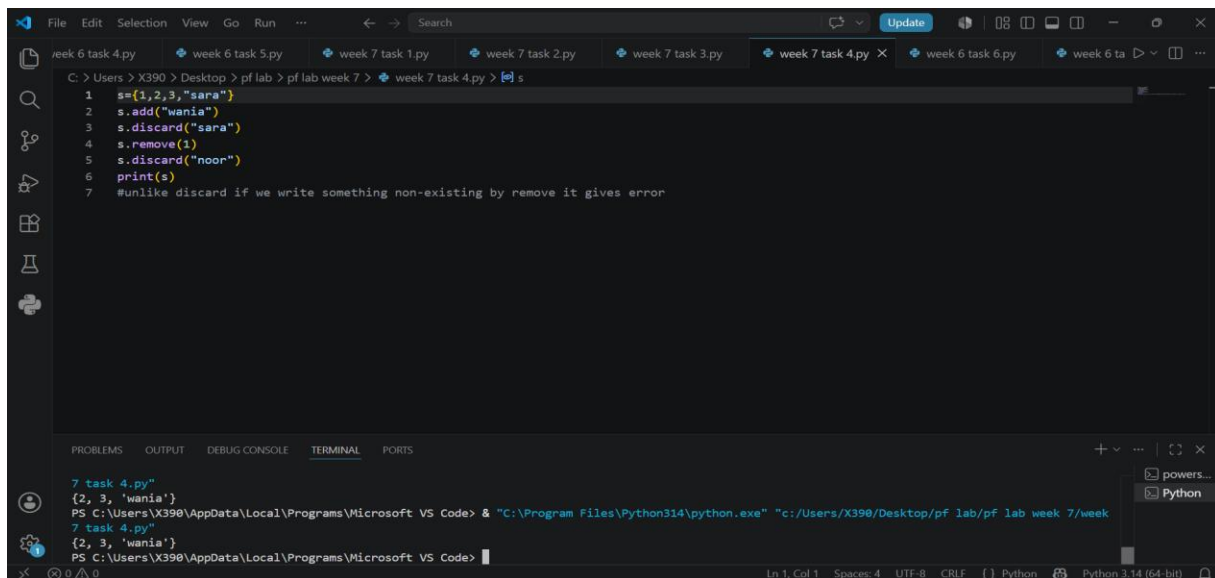
```
1 name={"zahra","ali","saba","zain"}
2 for i in name:
3     user = input("enter your name:")
4     if user in name:
5         print("exist")
6     else:
7         print("not exist")
```

7 task 3.py
enter your name:ZAHRA
not exist
enter your name:ALI
not exist
enter your name:

TASK NO 4:

Write a program that demonstrates how to insert and delete elements within a Python set structure, contrasting the behavioral differences between safe removal methods and error-prone removal methods.

RESULT:



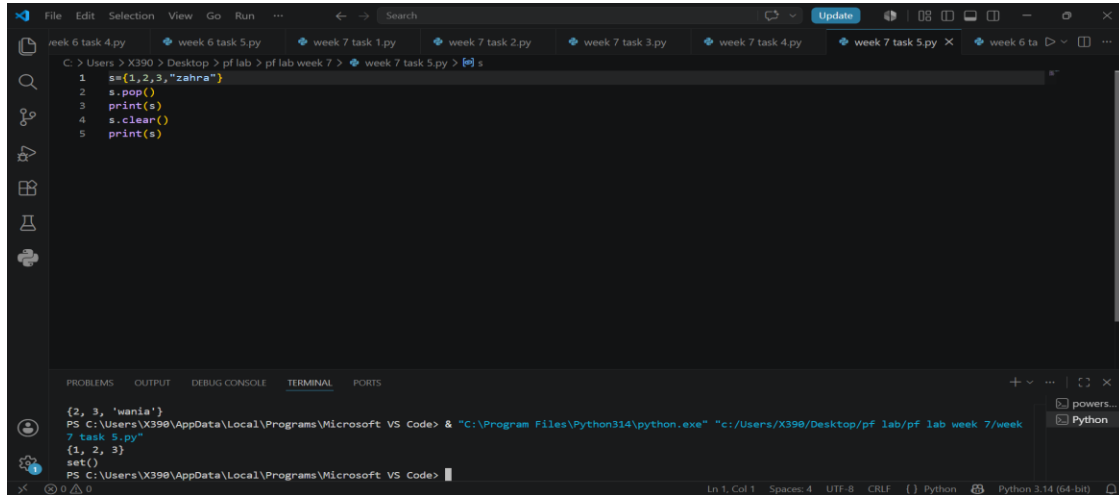
```
1 s={1,2,3,"sara"}
2 s.add("wania")
3 s.discard("sara")
4 s.remove(1)
5 s.discard("noon")
6 print(s)
7 #unlike discard if we write something non-existing by remove it gives error
```

7 task 4.py
{2, 3, 'wania'}
PS C:\Users\X398\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" "c:/Users/X398/Desktop/pf lab/pf lab week 7/week 7 task 4.py"
{2, 3, 'wania'}
PS C:\Users\X398\AppData\Local\Programs\Microsoft VS Code>

TASK NO 5:

Write a program that demonstrates how to manipulate and empty a Python set using the dynamic `. pop()` and `. clear()` built-in methods.

RESULT:



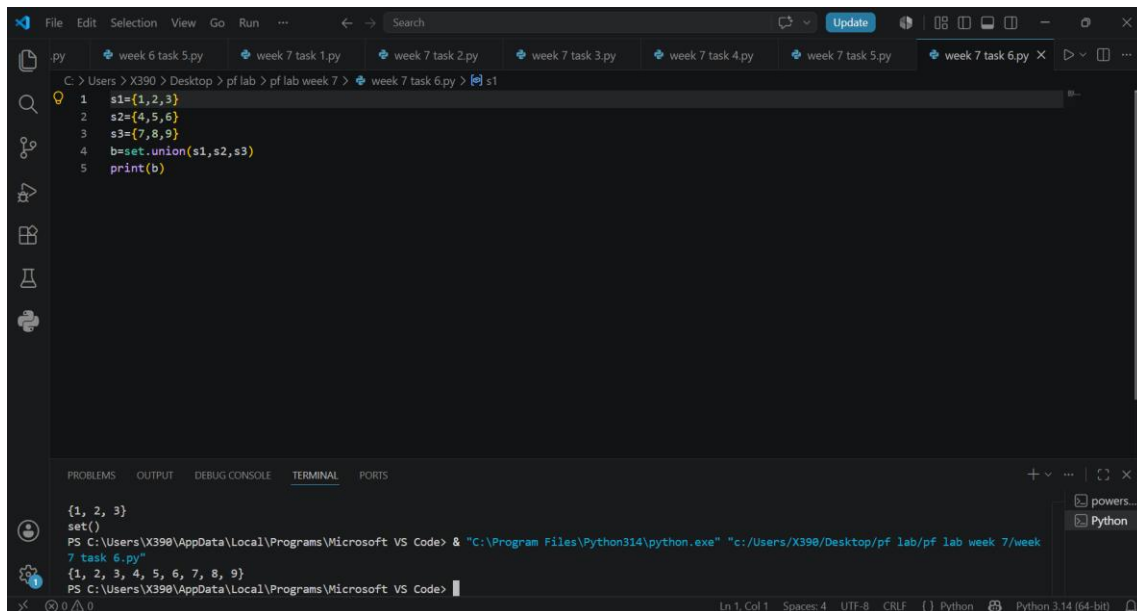
```
File Edit Selection View Go Run ... Search
week 6 task 4.py week 6 task 5.py week 7 task 1.py week 7 task 2.py week 7 task 3.py week 7 task 4.py week 7 task 5.py week 6 ta ...
C:\Users\X390\Desktop> pf lab > pf lab week 7 > week 7 task 5.py > s
1 s={1,2,3,"zahra"}
2 s.pop()
3 print(s)
4 s.clear()
5 print(s)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf lab/pf lab week 7/week
7 task 5.py"
{1, 2, 3}
set()
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code>
```

TASK NO 6:

Write a program that demonstrates how to perform a mathematical union operation across multiple independent set collections simultaneously in Python

RESULT:



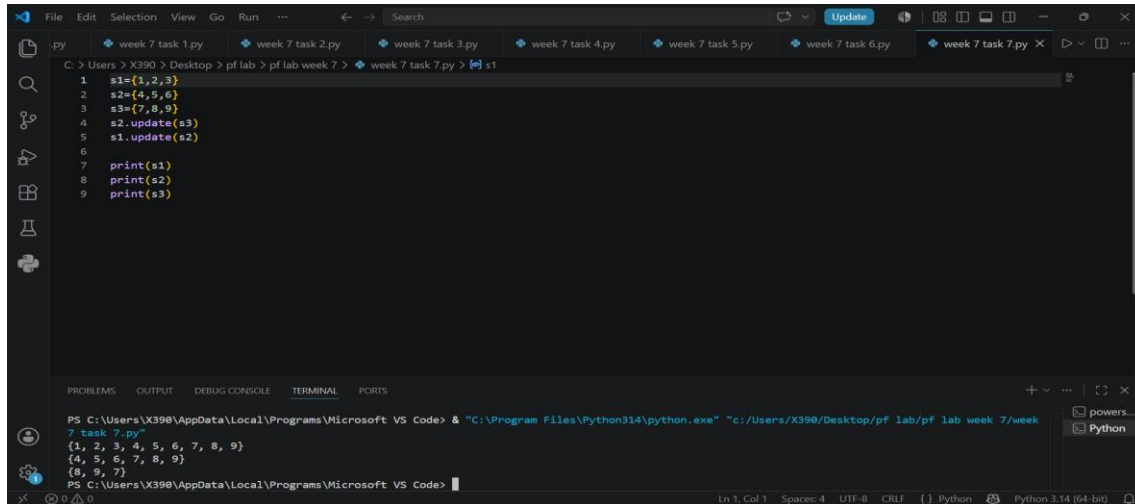
```
File Edit Selection View Go Run ... Search
.py week 6 task 5.py week 7 task 1.py week 7 task 2.py week 7 task 3.py week 7 task 4.py week 7 task 5.py week 7 task 6.py
C:\Users\X390\Desktop> pf lab > pf lab week 7 > week 7 task 6.py > s1
1 s1={1,2,3}
2 s2={4,5,6}
3 s3={7,8,9}
4 b=set.union(s1,s2,s3)
5 print(b)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
{1, 2, 3}
set()
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf lab/pf lab week 7/week
7 task 6.py"
{1, 2, 3, 4, 5, 6, 7, 8, 9}
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code>
```

TASK NO 7:

Write a program that demonstrates how to perform sequential, in-place union modifications on existing set collections using the built-in `.update()` method

RESULT:



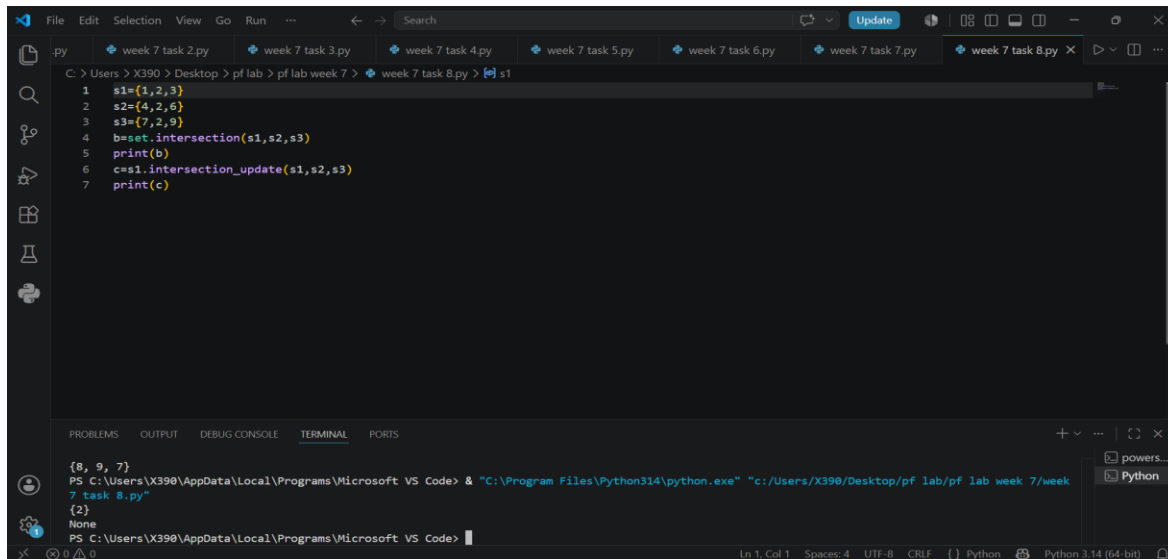
```
1 s1={1,2,3}
2 s2={4,5,6}
3 s3={7,8,9}
4 s2.update(s3)
5 s1.update(s2)
6
7 print(s1)
8 print(s2)
9 print(s3)
```

```
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf lab/pf lab week 7/week
7 task 7.py"
{1, 2, 3, 4, 5, 6, 7, 8, 9}
{4, 5, 6, 7, 8, 9}
{8, 9, 7}
```

TASK NO 8:

Write a program that demonstrates how to identify common matching elements across multiple Python set collections using the `set.intersection()` method and explores the behavior of `intersection_update()`.

RESULT:



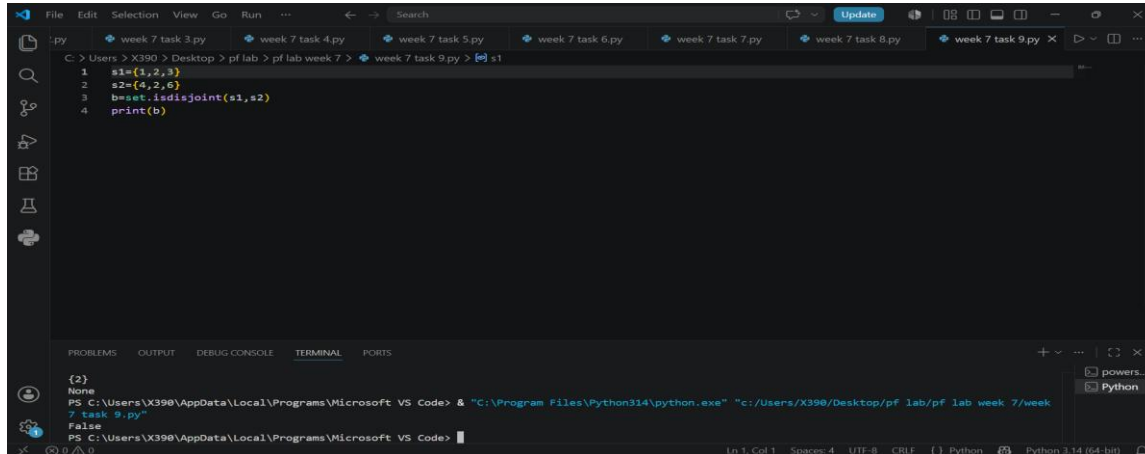
```
1 s1={1,2,3}
2 s2={4,2,6}
3 s3={7,2,9}
4 b=set.intersection(s1,s2,s3)
5 print(b)
6 c=s1.intersection_update(s1,s2,s3)
7 print(c)
```

```
{2}
None
```

TASK NO 9:

Write a program that uses the built-in `set.isdisjoint()` method to determine whether two specific set collections share any common elements

RESULT:



The screenshot shows the Visual Studio Code editor with a Python file named `task 9.py`. The code defines two sets, `s1 = {1, 2, 3}` and `s2 = {4, 2, 6}`, and uses the `set.isdisjoint()` method to check if they share any common elements. The result is stored in variable `b` and printed. The terminal output shows the execution of the script, resulting in `False`.

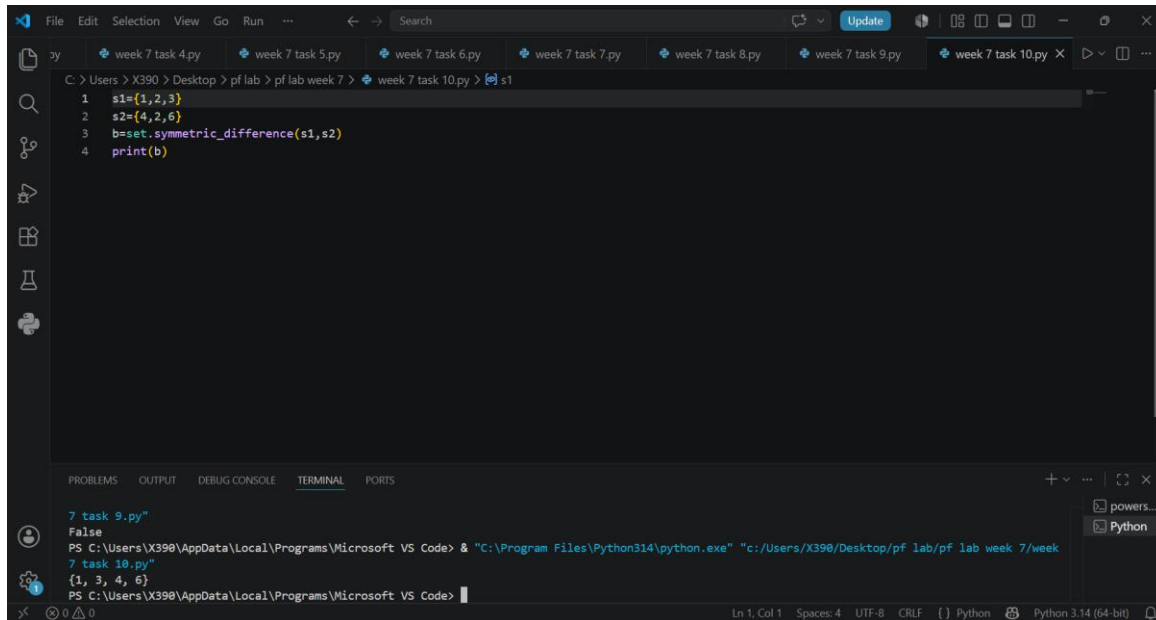
```
1 s1={1,2,3}
2 s2={4,2,6}
3 b=set.isdisjoint(s1,s2)
4 print(b)
```

```
PS C:\Users\X390\Desktop> cd pf lab & cd pf lab week 7 & cd task 9.py & python s1
False
```

TASK NO 10:

Write a Python program to find and display the **symmetric difference** between two sets of integers.

RESULT:



The screenshot shows the Visual Studio Code editor with a Python file named `task 10.py`. The code defines two sets, `s1 = {1, 2, 3}` and `s2 = {4, 2, 6}`, and uses the `set.symmetric_difference()` method to find the symmetric difference between them. The result is stored in variable `b` and printed. The terminal output shows the execution of the script, resulting in `{1, 3, 4, 6}`.

```
1 s1={1,2,3}
2 s2={4,2,6}
3 b=set.symmetric_difference(s1,s2)
4 print(b)
```

```
PS C:\Users\X390\Desktop> cd pf lab & cd pf lab week 7 & cd task 10.py & python s1
{1, 3, 4, 6}
```

CONCLUSION:

- The lab exercises successfully demonstrated the fundamental differences in behavior, memory utilization, and syntax between mutable and immutable Python data structures. Lists provided the flexibility required for dynamic operations like appending and removing elements, while tuples offered strict data integrity by preventing in-place modifications. Dictionaries further expanded on these structures by demonstrating highly efficient key-value lookups and flexible, real-time data mapping capabilities. Implementing these collection types entirely within the global scope ensured maximum data availability across each individual script's execution lifecycle.